CSCI3180 Principles of Programming Languages

Ryan Chan

May 2, 2026

Abstract

This is a note for **CSCI3180 Principles of Programming Languages**.
Contents are adapted from the lecture notes of CSCI3180, prepared by Lauren Pick, as well as some online resources.
This note is intended solely as a study aid. While I have done my best to ensure the accuracy of the content, I do not take responsibility for any errors or inaccuracies that may be present. Please use the material thoughtfully and at your own discretion.
If you believe any part of this content infringes on copyright, feel free to contact me, and I will address it promptly.
Mistakes might be found. So please feel free to point out any mistakes.

Contents

1 Foundations **2**
1.1 Syntax and Semantics 2
1.2 Formal Syntax Rules 2
1.3 Abstract syntax trees (ASTs) 5
1.4 Beyond BNF and Meta-variables 6

Chapter 1

Foundations

1.1 Syntax and Semantics

Consider an arbitrary programming language $\mathcal{L}$ (a set of programs). We have:

- **Syntax**: describes which things are programs in $\mathcal{L}$

- **Semantics**: describes what happens when we run a program in $\mathcal{L}$

So, given the syntax for a programming language $\mathcal{L}$, with a set of values $\mathcal{V}$, we define

$$\mathsf{Interpreter}_{\mathcal{L}} : \mathcal{L} \to \mathcal{V}$$

Here the *interpreter* takes a program in $\mathcal{L}$ and runs it to produce a value in $\mathcal{V}$. For example,

$$\mathsf{Interpreter}_{\text{Python}}(\texttt{sorted([1, 4, 3, 2])}) = \texttt{[1, 2, 3, 4]}$$

This can be seen as one way of defining the semantics of the program.

A compiler translates from one language to another. For a program p in $\mathcal{L}_1$ and $\mathsf{Compiler} : \mathcal{L}_1 \to \mathcal{L}_2$ to be correct, we want

$$\mathsf{Interpreter}_{\mathcal{L}_1}(p) = \mathsf{Interpreter}_{\mathcal{L}_2}(\mathsf{Compiler}(p))$$

otherwise something is incorrect.

Notice that interpreters and compilers, in **theory**, are mathematical objects, but not in reality. They have implementations, and while we can treat implementations of programming languages as defining their semantics, it is helpful not to do this, since implementations of interpreters and compilers are themselves programs and can be buggy.

Furthermore, different implementations of the same programming language may exhibit *diverging behaviors*, especially in the presence of under-specification. We can see that it is helpful to know what programs are meant to do (i.e., their semantics) separately from their implementations.

1.2 Formal Syntax Rules

1.2.1 Formal Language

A formal language L over an alphabet Σ is a subset of Σ^*, which is the set of strings consisting of and making use of elements in Σ. The empty string is denoted by ε.

For example, a formal language L over $\Sigma = \{1, 2\}$ is a subset of Σ^*, e.g. $L = \{\varepsilon, 1, 2, 11, 22, 111, 222\}$.

One way to manipulate strings is using a rewriting system, which consists of rules that map strings in a formal language to other strings in the language. For a language L, we define a relation $\to$ over strings in L. For example, for $L = \{1, 2, 11\}$, we may have $\to = \{(1, 2), (2, 1), (1, 11), (11, 2)\}$. Then we can write $1 \to 2$, $2 \to 1$, $1 \to 11$, and $11 \to 2$.

We also have generative grammar, which is a kind of rewriting system over a language $(\Sigma \cup N)^*$, where

- the alphabet is partitioned into two sets: a set of non-terminal symbols N and a set of terminal symbols Σ,

- there is a distinguished start symbol in N,

- rewrite rules are called *production rules*; the left-hand side of each rule contains at least one non-terminal.

We can use generative grammars to define a language over the alphabet Σ. The language defined is the set of strings of terminal symbols that we can generate by starting with the start symbol S and following the production rules.

Example (Generative Grammar). Consider $\Sigma \equiv \{a, b, c\}$.

Language ONE with only two strings $\{a, b\}$.

In this case, to generate a language over Σ, we have $N = \{S\}, \Sigma = \{a, b, c\}$ with grammar (production rules):

$$S \to a$$
$$S \to b$$

Language A-B where no a follows any b, e.g. $\varepsilon, ab, aab, acb$.

In this case, consider $N = \{S, A, B, C\}, \Sigma = \{a, b, c\}$ with production rules:

$$\begin{array}{ll} S \to CABC & \\ C \to \varepsilon & C \to cC \\ A \to \varepsilon & A \to aCA \\ B \to \varepsilon & B \to bCB \end{array}$$

Language MATCH where the number of as and bs is the same.

In this case, consider $N = \{S\}, \Sigma = \{a, b, c\}$ with production rules:

$$\begin{array}{ll} S \to \varepsilon & S \to c \\ S \to aSb & S \to bSa \\ S \to SS & \end{array}$$

$$\begin{tikzcd} & S \arrow[dl, "S \to a"'] \arrow[dr, "S \to b"] & \\ a & & b \end{tikzcd}$$

$$\begin{tikzcd} S \arrow[d, "S \to CABC"'] & & \\ CABC \arrow[d, "C \to \varepsilon"'] & & \\ ABC \arrow[d, "A \to \varepsilon"'] & & C \arrow[d, "C \to \varepsilon"] \\ BC \arrow[urr, "B \to \varepsilon"] & & \varepsilon \end{tikzcd}$$

1.2.2 Context-Free Grammar (CFGs)

The above grammars are context-free grammars, where the left-hand side of each production rule consists of a single non-terminal. It is often specified using Backus-Naur Form (BNF), where

- every production rule is written with ::= instead of $\to$,

- production rules with the same left-hand side are combined into a single rule, where different right-hand sides are separated with |, e.g. Language BOOL of boolean formulas with

- conjunction (and) $\wedge$
- disjunction (or) $\vee$
- negation (not) $\neg$

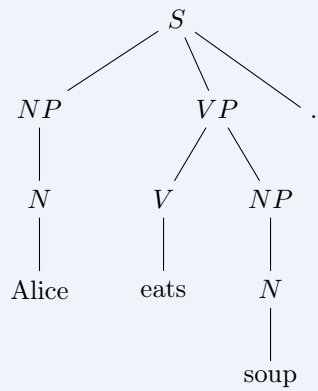
$$S ::= x \mid y \mid z \mid w \mid (S) \mid S \wedge S \mid S \vee S \mid \neg S$$

Example (BNF). Consider a BNF grammar for LUNCH$^-$.

$$\begin{aligned}
S &::= NP\ VP. \\
NP &::= N \mid N\ PP \\
VP &::= V \mid V\ NP \mid V\ PP \mid V\ NP\ PP \\
V &::= \text{buys} \mid \text{eats} \\
N &::= \text{Alice} \mid \text{Bob} \mid \text{soup} \mid \text{salad} \mid \text{pizza} \\
PP &::= \text{in a bowl} \mid \text{in a teacup} \mid \text{in a house}
\end{aligned}$$

Then we can derive a derivation tree (parse tree):

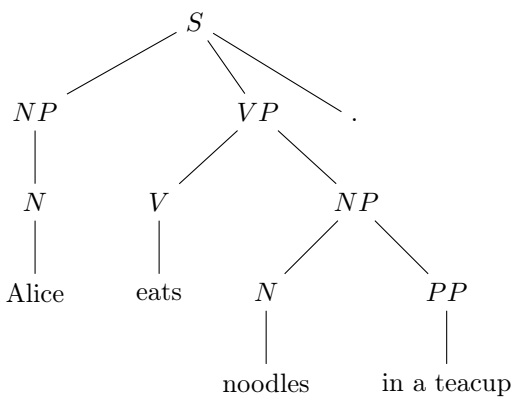
$$\begin{aligned}
S &\to NP\ VP. \\
&\to N\ VP. \\
&\to \text{Alice}\ VP. \\
&\to \text{Alice}\ VNP. \\
&\to \text{Alice eats}\ NP. \\
&\to \text{Alice eats}\ N. \\
&\to \text{Alice eats soup.}
\end{aligned}$$

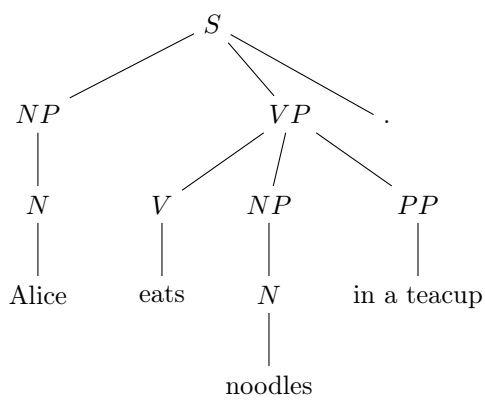


Remark. A parse tree acts as a proof that a string is in a language (membership).

1.2.3 Ambiguity & *Dangling Else Problem*

A grammar is ambiguous if the same string has more than one parse tree, e.g.





Grammar serves an additional purpose, that is, it can be used to give structure to a string in a language. Semantics for the strings within a language are typically described with respect to this additional structure imposed on the string. Ambiguous grammar can thus lead to ambiguous meanings being assigned to strings.

For example, in the above examples, the left suggests that the noodles are in a teacup, whereas the right one suggests that Alice is located in a teacup. While one interpretation may be more likely than another and judgment can be based on semantics and background knowledge, the correct conclusion is not determined by the grammar alone.

This problem can manifest in the *dangling else problem* in a programming language context.

Consider language IF with the following grammar:

$$\begin{aligned} S &::= C \mid \text{ if } B \text{ then } S \mid \text{ if } B \text{ then } S \text{ else } S \\ B &::= \text{ true } \mid \text{ false } \mid \ldots \\ C &::= \ldots \end{aligned}$$

To deal with ambiguity, we can

1. change the grammar such that it is **not** ambiguous, i.e. same language with a different grammar.

For example, for BOOL:

$$S ::= x \mid y \mid z \mid w \mid (S) \mid S \wedge S \mid S \vee S \mid \neg S$$

we change to

$$\begin{aligned} S &::= D \vee S \mid D \\ D &::= C \wedge D \mid C \\ C &::= (S) \mid \neg C \mid x \mid y \mid z \mid w \end{aligned}$$

2. change the syntax, i.e. a different language with a different grammar.

For example, we change language IF to

$$S ::= C \mid \text{ if } B \text{ then } S \text{ end if } \mid \text{ if } B \text{ then } S \text{ else } S \text{ end if}$$

1.3 Abstract syntax trees (ASTs)

Before, we have $\mathsf{Interpreter}_{\mathcal{L}}$ for $\mathcal{L}$ that takes a program in $\mathcal{L}$ and produces a value. Typically, there is a frontend containing a *parser* and a backend containing an *evaluator*. The parser parses the input string according to a grammar and converts the input program into an intermediate representation called an abstract syntax tree (AST),

$$\mathsf{Parser}_{\mathcal{L}} : \mathcal{L} \to \mathrm{AST}(\mathcal{L})$$

and the evaluator then evaluates the AST to a value,

$$\mathsf{Evaluator}_{\mathrm{AST}(\mathcal{L})} : \mathrm{AST}(\mathcal{L}) \to \mathcal{V}$$

Then we have

$$\mathsf{Interpreter}_{\mathcal{L}} = \mathsf{Evaluator}_{\mathrm{AST}(\mathcal{L})} \circ \mathsf{Parser}_{\mathcal{L}}$$

The AST representation abstracts a parse tree and keeps only the relevant syntactic information from the parse tree, in particular, a subset of the production rules applied to obtain the parse tree.

To make an AST:

1. derive a parse tree annotated with (important) production rules applied,

2. remove terminal nodes,

3. remove nodes expanded by production rules we do not care about; on removal, parent the node's children to its parent,

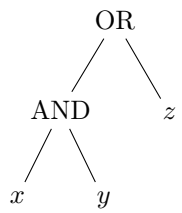
4. relabel non-terminal nodes with the annotated production rule names.

For example, for $x \wedge y \vee z$ with

$$\begin{aligned} S &::= D \vee S \mid D \\ D &::= C \wedge D \mid C \\ C &::= (S) \mid \neg C \mid x \mid y \mid z \mid w \end{aligned}$$

we have

finally we have



Notice that we have viewed ASTs as generic trees, but they are often trees with a specific structure, i.e. nodes with a certain label might always have a particular degree or children that have only certain kinds of labels. This structure can be captured by another grammar, where we introduce a production rule in the AST grammar for each production rule that we care about. For example, for the above BOOL, we might have the following grammar:

$$AST ::= OR(AST, AST) \mid AND(AST, AST) \mid NOT(AST) \mid x \mid y \mid z \mid w$$

This can be directly translated into an ADT, such as the following in OCaml:

```
type bool_ast = Or of bool_ast * bool_ast
  | And of bool_ast * bool_ast
  | Not of bool_ast
  | X
  | Y
  | Z
  | W
```

The `bool_ast` for $x \wedge y \vee z$ is `Or(And(X,Y),Z)`.

When implementing a parser, rather than generating a full parse tree and then an AST, we can generate the AST during parsing by applying the appropriate ADT constructor for a production rule.

1.4 Beyond BNF and Meta-variables

Many programming languages use formal grammars to specify syntax, yet they do not directly use BNF. Instead, they use **extensions** or **variants** of BNF.

For example, in the language BOOL, we have

$$Vars \cup \{\wedge, \vee, \neg, (,)\}$$

and in the grammar each variable in $Vars$ becomes a terminal in the production rules. To extend BOOL such that it allows more variables requires updating the BNF grammar with new production rules. However, when we want to allow infinitely many variable names, this becomes impossible as BNF grammars are finite.

We then use extensions of BNF which allow using additional notations outside of BNF, e.g. *meta-variables*. For example, let $BoolVars$ be an infinite set of variables and let v range over $BoolVars$; then v is a stand-in for any element of $BoolVars$. Then we can still construct parse trees using BNF-like grammars and derive ASTs for them as before.

The OCaml ADT for the structure of the ASTs for BOOL$^+$ could be something like

```
type bool_ext_ast = Or of bool_ext_ast * bool_ext_ast
  | And of bool_ext_ast * bool_ext_ast
  | Not of bool_ext_ast
  | Var of string
```